MusicInsta

PROJECT SUBMISSION GUIDE

MusicInsta

Project README

A branded guide to the platform, architecture, recommendation engine, local setup and assessment environment.

RECOMMENDED DEMONSTRATION

artist-amp.vercel.app

Prosper Masuku - Dby Prosper

Computer Science Honours Research Project

University of Fort Hare

MusicInsta is a social music platform created for independent artists and listeners. It combines music publishing, personalised discovery, social interaction, artist profiles, playlists, an online digital audio workstation (DAW), and AI-assisted music-production tools in one application.

Recommended way to access the project

For the quickest and easiest assessment, use the deployed application:

Open MusicInsta on Vercel

The hosted version is recommended because it is already connected to the deployed API, Firebase services, Firestore database, and Google Cloud Storage media catalogue. Running locally requires Node.js, Firebase configuration, Google Cloud credentials, Python dependencies, and audio-processing tools.

Main features

+ Artist and listener registration, authentication, profiles, preferences, and following.
+ Music, image, video, album, story, caption, hashtag, mention, like, comment, reply, and messaging features.
+ Personalised home feed and music discovery by genre, listening history, likes, skips, completions, mood, tags, and track similarity.
+ Global music player with queues, recommendations, lyrics, playlists, play counts, and DAW transfer for owned music.
+ Browser-based DAW with recording, arranging, mixing, effects, project saving, and cloud synchronisation.
+ AI Studio tools for beats, full songs, lyrics, cover art, posters, merchandise, audio enhancement, stem separation, and AI chat.
+ Administrative tools for posts, users, AI availability, quotas, and the official MusicInsta account.
+ Responsive landing, feed, discovery, studio, profile, and administrative interfaces.

Technology overview

Area	Technologies
Frontend	React, TypeScript, Vite, Tailwind CSS, shadcn/ui, Framer Motion
Authentication and data	Firebase Authentication and Cloud Firestore
Media	Google Cloud Storage
Backend	Python, FastAPI, Uvicorn
AI and audio	Gemini, Vertex AI, Lyria, Demucs, FFmpeg and supporting audio libraries
Hosting	Vercel frontend and Google Cloud Run backend

Project structure

The submission contains two related projects:

```
artist-amp/ React frontend
|-- src/components/ Reusable UI, feed, player and studio components
|-- src/context/ Authentication and global player state
|-- src/pages/ Application pages
|-- src/services/ Catalogue and recommendation services
|-- src/test/ Frontend tests
|-- firestore.rules Firestore security rules
`-- CONTENT_BASED_RECOMMENDATION_ALGORITHM.txt

musicinsta-ai/ FastAPI backend
|-- app/audio/ Audio enhancement and separation routes
|-- app/core/ Quotas and shared backend controls
|-- app/routes/ API route modules
|-- app/services/ AI, media and recommendation services
|-- tests/ Backend tests
|-- requirements.txt Python dependencies
`-- Dockerfile Cloud Run container definition
```

Recommendation algorithm

The production content-based recommendation algorithm is located at:

+ `artist-amp/src/services/recommendationEngine.ts` - similarity calculation, listener profiling, catalogue ranking, feed ranking, artist diversification, and personalised queue construction.
+ `artist-amp/src/services/musicCatalogService.ts` - loads the playable Firestore-indexed Google Cloud Storage catalogue and listening history.
+ `artist-amp/src/context/PlayerContext.tsx` - connects recommendations to the global music player and queue.
+ `artist-amp/src/pages/HomePage.tsx` and `src/pages/DiscoverPage.tsx` - connect personalised ranking to the feed and discovery page.
+ `musicinsta-ai/app/services/recommendation_service.py` and `app/routes/recommendation.py` - backend catalogue and recommendation API.
+ `artist-amp/CONTENT_BASED_RECOMMENDATION_ALGORITHM.txt` - isolated plain-text explanation of the algorithm for assessment.

The algorithm builds weighted track vectors from genre, mood, artist genres, hashtags/tags, title, lyrics, BPM, energy, and loudness. It combines content similarity with the listener's preferred genres and private behaviour, including plays, completed listens, likes, and skips. Familiar matching music is prioritised first, followed by diversified recommendations from other artists. New posts remain near the top of the feed, while older music posts are personalised. Recommendations come from the live Firestore and Google Cloud Storage catalogue rather than mock data or a local CSV file.

Run the frontend locally

Requirements

+ Node.js 20 or newer
+ npm
+ A Firebase web application configuration
+ The deployed backend URL, or a locally running backend

Setup

Open PowerShell in the submitted `artist-amp` folder:

```
npm install
Copy-Item .env.example .env
```

Edit `.env` and supply the Firebase web configuration. To use the deployed backend, keep:

```
VITE_API_BASE_URL=https://musicinsta-ai-973497466485.us-central1.run.app
```

Start the application:

```
npm run dev
```

Open http://localhost:8080.

Useful verification commands:

```
npm test
npm run build
```

Run the backend locally

The hosted frontend already uses the deployed Cloud Run backend, so running the backend locally is optional. For full local development, open PowerShell in the submitted `musicinsta-ai` folder.

Additional requirements

+ Python 3.10 or 3.11
+ FFmpeg available on the system path
+ Google Cloud project access and application-default credentials or a service-account file
+ Gemini/Google AI credentials required by the selected generation tools

Setup

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
Copy-Item .env.example .env
```

Edit `.env` with the appropriate project, bucket, and API credentials. Never commit service-account JSON files or real API keys.

Start the API on port `8000` so it can run beside the frontend:

```
$env:PORT="8000"
python main.py
```

The API runs at http://localhost:8000, and its interactive documentation is available at http://localhost:8000/docs.

When running both projects locally, set `VITE_API_BASE_URL=http://localhost:8000` in the frontend `.env` and restart the frontend development server.

Run the backend tests with:

```
python -m pytest tests -q
```

Important notes for assessment

+ The deployed Vercel URL is the recommended demonstration environment because local AI and audio processing require large dependencies and authorised cloud credentials.
+ Some generation tasks, especially full-song generation and stem separation, can take several minutes.
+ AI Studio usage is quota-controlled for normal users; administrators have separate access controls.
+ Guests can browse and play public content, but publishing, social interactions, playlists, DAW usage, and AI generation require authentication.
+ Generated and uploaded media are stored in Google Cloud Storage and indexed in Firestore so they can be discovered, recommended, and played throughout the application.

Author

Prosper Masuku - Dby Prosper

Computer Science Honours research project, University Of Fort Hare.

For the fastest assessment experience, open the deployed application at https://artist-amp.vercel.app.